

Tutorial Configuración Ambientes de prueba: QA, Sandbox, preprod y prod

Paso1: Se tiene que crear esta estructura para el proyecto.

.github/

└─ workflows/

├─ sandbox.yml

├─ qa.yml

├─ preprod.yml

└─ prod.yml

Cada archivo representa el **pipeline de un ambiente**.

Paso 2. En cada .yml configurar el contenido.

- Sandbox
name: CI - Sandbox

on:
 push:
 branches:
 - nombre rama

jobs:
 build:
 runs-on: ubuntu-latest

 steps:
 - name: Checkout repository
 uses: actions/checkout@v4

 - name: Set up JDK 21
 uses: actions/setup-java@v4
 with:
 distribution: 'temurin'
 java-version: '21'

 - name: Build and test (Sandbox)

```
run: |  
  echo "Running build for SANDBOX environment"  
  mvn clean verify
```

- name: Simulate deploy to Sandbox

```
run: |  
  mkdir -p ./deploy/sandbox  
  cp -r ./target ./deploy/sandbox  
  echo " App deployed to Sandbox (simulated)"
```

Este ambiente es para pruebas de desarrollo rápidas.

- Qa

name: CI - QA

on:

push:

branches:

- nombre rama

jobs:

test:

runs-on: ubuntu-latest

steps:

- name: Checkout repository
uses: actions/checkout@v4

- name: Set up JDK 21
uses: actions/setup-java@v4
with:
 distribution: 'temurin'
 java-version: '21'

- name: Run QA tests

```
run: |  
  echo "Running QA tests (unit + integration)"  
  mvn clean verify
```

```
- name: Simulate deploy to QA
run: |
  mkdir -p ./deploy/qa
  cp -r ./target ./deploy/qa
  echo "App deployed to QA environment (simulated)"
```

Este ambiente es para validaciones funcionales por el equipo de QA.

- Preprod

name: CI - Preproduction

on:

push:

branches:

- nombre rama

jobs:

integration:

runs-on: ubuntu-latest

steps:

- name: Checkout repository

uses: actions/checkout@v4

- name: Set up JDK 21

uses: actions/setup-java@v4

with:

distribution: 'temurin'

java-version: '21'

- name: Build and integration tests

run: |

echo "Running integration and performance tests"

mvn clean verify -Pintegration

- name: Simulate deploy to Preprod

run: |

mkdir -p ./deploy/preprod

cp -r ./target ./deploy/preprod

```
echo " App deployed to Preproduction (simulated)"
```

Este ambiente simula producción antes del despliegue real.

- Prod
name: CI/CD - Production

on:
push:
branches:
- nombre rama

jobs:
release:
runs-on: ubuntu-latest

steps:
- name: heckout repository
uses: actions/checkout@v4

- name: Set up JDK 21
uses: actions/setup-java@v4
with:
distribution: 'temurin'
java-version: '21'

- name: Build for production
run: |
echo "Building production release"
mvn clean package -Pprod

- name: Simulate deploy to Production
run: |
mkdir -p ./deploy/production
cp -r ./target ./deploy/production
echo "Production build simulated successfully"

Este es el ambiente estable, la versión final que será desplegada en Azure más adelante.

Cada uno ejecutará automáticamente el build, las pruebas y un despliegue simulado cuando se haga un **push a la rama correspondiente**.

Verificación: Ve a **GitHub → Actions** y deberías ver los **4 workflows ejecutándose en paralelo**, todos activados por el push.

Paso 3. Crear los Environments

En GitHub Settings → Environments toca crear un Environment por ambiente de prueba que serán llamados de la siguiente manera

- sandbox
- qa
- preprod
- production

Dentro de cada Environments se deben crear los siguientes Secrets

- DATABASE_URL
- SPRING_PROFILES_ACTIVE
- APP_SECRET
- AZURE_WEBAPP_NAME
- AZURE_PUBLISH_PROFILE

De momento los dejamos vacíos

Paso 4. Conexión de Azure

Paso 1.

En App Service toca darle en +Crear y en application web, en este solo se debe llenar los siguientes espacios:

- Grupo de recursos: definido
- Nombre: definido
- Pila de Entorno en tiempo de ejecución: Java 21 o Java 17(No importa luego puede cambiar de versión)
- Plan de precios: F1 Gratis

Luego darle en crear

Un **App Service** por ambiente:

- sirha-sbx
- sirha-qa

- sirha-preprod
- sirha-pro

Paso 2. Conectar al GitHub

En centro de implementaciones:

- Origen: GitHub
- Sesión iniciada como: Inicie con su cuenta
- Organización: Su usuario de GitHub
- Repositorio: El repositorio en el que trabaja
- Rama: La rama donde quiere desplegar}

Luego dele a guardar y se conectara.

Paso 5 configurar la BD de cosmos

Paso 1. Crear Bd

Entrar a AZURE Cosmo DB for MongoDB(vCore) y darle a crear.

Nombre:

- sirha-sbx-db
- sirha-qa-db
- sirha-preprod-db
- sirha-prod-db

En ello se llenan los siguientes campos

- Resource group: definido
- Cluster name: definido
- Cluster tier -> configure -> Free tier
- Admin username: definido
- Password: definido
- Confirm password: definido

Siguiente en Firewall rules darle a + Add 0.0.0.0 - 255.255.255.255 y hay darle a create.

Paso 6. llenar los secrets

En los secrets dentro de cada Environment debe llenarse así por cada Ambiente de prueba.

Nombre	Valor (ejemplo)
DATABASE_URL	En el App service entra a Cadena de conexión y copia el que diga “Global read-write connection string (currently this cluster)” y pegar aquí Nota: en el link aparece <password> debe quitar eso y poner la contraseña que definió.
SPRING_PROFILES_ACTIVE	sandbox / qa / preprod / prod
AZURE_WEBAPP_NAME	nombre exacto del App Service
AZURE_PUBLISH_PROFILE	En Introducción en (...) dale a Descargar perfil de publicación abre el archivo y pégalo aquí. Nota: Si no deja descargarlo ve a configuraciones y activa Credenciales de publicación de autenticación básica de SCM
APP_SECRET	sirha-secret-key
MONGO_DB_NAME	sandbox / qa / preprod / prod dependiento como quiera llamar la BD

Luego actualiza el application.properties con la siguiente implementación para poder utilizar los secretos configurados:

```
### Conexion base de datos MongoDB (No relacional)
spring.data.mongodb.uri=${DATABASE_URL:mongodb+srv://JuanCaballero:Isabella2004@clustersirha.pc6me39.mongodb.net/?retryWrites=true&w=majority&appName=ClusterSirha}
spring.data.mongodb.database=${MONGO_DB_NAME:SIRHA}
```

```
### Variables de entorno (usadas en GitHub Actions / Azure)
spring.profiles.active=${SPRING_PROFILES_ACTIVE:SIRHA}
server.port=${PORT:8080}
```

Luego cambia los .yml de cada Ambiente de prueba por similares al de sandbox:

name: CI/CD - sandbox

on:

push:

branches:

- "*"

workflow_dispatch:

jobs:

build:

name: Build and Test

runs-on: ubuntu-latest

environment: sandbox

steps:

- name: Checkout code

uses: actions/checkout@v4

- name: Set up JDK 17

uses: actions/setup-java@v4

with:

java-version: '17'

distribution: 'temurin'

- name: Build with Maven

run: mvn clean install -DskipTests=false

- name: Upload build artifact

uses: actions/upload-artifact@v4

with:

name: java-app

path: target/*.jar

deploy:

name: Deploy to Azure

runs-on: ubuntu-latest

needs: build

environment: sandbox

env:

SPRING_PROFILES_ACTIVE: \${{ secrets.SPRING_PROFILES_ACTIVE }}

DATABASE_URL: \${{ secrets.DATABASE_URL }}

MONGO_DB_NAME: \${{ secrets.MONGO_DB_NAME }}

APP_SECRET: \${{ secrets.APP_SECRET }}

steps:

- name: Download build artifact

uses: actions/download-artifact@v4

with:

name: java-app

- name: Azure WebApp Deploy

uses: azure/webapps-deploy@v3

with:

app-name: \${{ secrets.AZURE_WEBAPP_NAME }}

```
publish-profile: ${ secrets.AZURE_PUBLISH_PROFILE }}
```

```
package: '*.jar'
```

Paso 7. Confirma conexión

Paso 8. Configurar los controllers para el API

Configura estos dos controladores con el siguiente propósito

Controlador	Ruta principal	Rol	Resultado
HomeController	/	Página de bienvenida	Evita 404 y guía al usuario
StatusController	/api/status	Prueba de conexión a la BD	Confirma conexión a Cosmos DB

Configuraciones básicas

- StatusController:

```
package eci.edu.dosw.proyecto;
```

```
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;  
import org.springframework.data.mongodb.core.MongoTemplate;  
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;
```

```
@RestController
```

```
@RequestMapping("/api")
```

```
public class StatusController {
```

```
    private static final Logger logger =  
    LoggerFactory.getLogger(StatusController.class);
```

```
    @Autowired
```

```
    private MongoTemplate mongoTemplate; // obligatorio, fallará al arrancar si  
    no se crea
```

```
    @GetMapping("/status")
```

```
    public ResponseEntity<String> checkStatus() {
```

```

    if (mongoTemplate == null) {
        logger.error("MongoTemplate no está inicializado!");
        return ResponseEntity.status(500).body(" MongoTemplate is not
initialized!");
    }

    try{
        // Intentamos listar colecciones para validar la conexión
        mongoTemplate.getDb().listCollectionNames().first();
        logger.info("Conexión a MongoDB (Cosmos) exitosa!");
        return ResponseEntity.ok(" Connected to MongoDB (Cosmos)
successfully!");
    } catch (Exception e) {
        logger.error("Error al conectar con MongoDB: {}", e.getMessage(), e);
        return ResponseEntity.status(500)
            .body("Connection error: " + e.getMessage());
    }
}
}

package eci.edu.dosw.proyecto.configs;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {

    @GetMapping("/")
    public String redirectToSwagger() {
        return "redirect:/swagger-ui/index.html";
    }
}

```

Configura también las mismas variables en Azure que en git actions, se realiza en configuraciones ->variables de entorno.

Para confirmar la conexión usa `mvn spring-boot:run` y verifica con alguno de estos dos métodos:

- <http://localhost:8080/api/status> En el link debe aparecer Connected to MongoDB (Cosmos) successfully para que este correcta la conexión.
- El git bash curl <http://localhost:8080/api/status> donde aparece Connected to MongoDB (Cosmos) successfully! para que este correcta la conexión.

Solo sirve localmente, es decir debes usar los valores que están en los secrets.